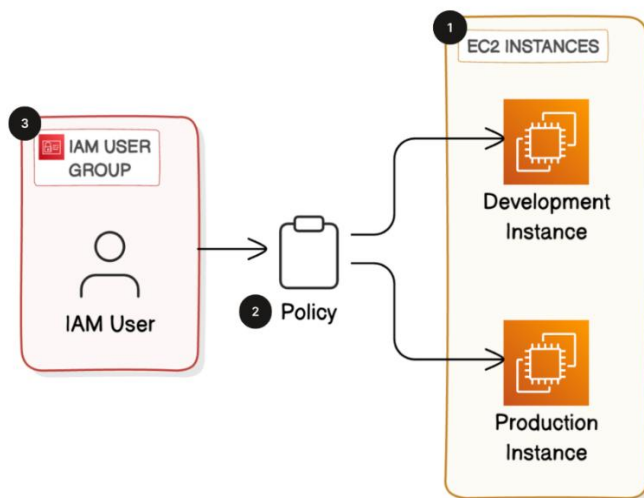

Cloud Security with AWS IAM

KEY CONCEPTS: AWS IAM, Amazon EC2

In AWS, a user is a person or a computer that can do things on the cloud - just like you right now!

Today, we'll be using the AWS Identity and Access Management (IAM) service to control who is authenticated (signed in) and authorized (has permissions) in your AWS account.

We'll launch an EC2 instance, then control who has access to it by creating some IAM policies and user groups. It will look something like this...



TASKS

1. 🖥️ EC2 instances
2. 📄 IAM Policies
3. 👥 IAM Users and User Groups
4. 🚀 AWS Account Alias

Launch EC2 Instances

As we gear up for the upcoming holiday season, we need to:

1. ⚡ **Boost our computing power** to match increased traffic to the website. Lots of new students want to learn with NextWork over the break!
2. 🧑 **Onboard an intern** working at NextWork - they should have the **right permission settings** to contribute while keeping the company's resources secure.

Let's start with the first task and **boost computing power** by launching some EC2 instances.

- [Log in to your AWS Management Console.](#)

- Open your EC2 console - search for it at the search bar.

WHAT IS EC2?

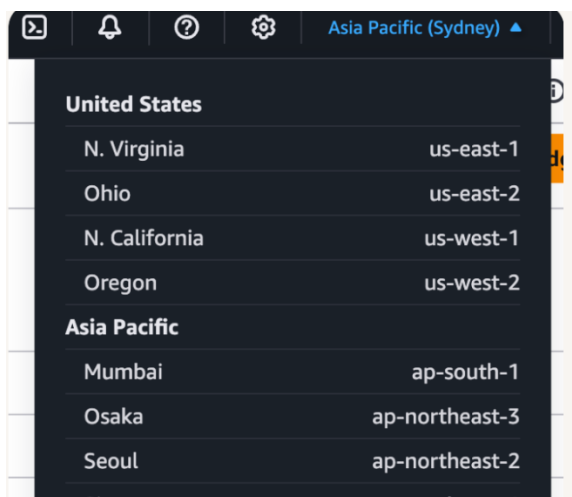
A legendary AWS service! Amazon EC2 is a service that lets you rent and use virtual computers in the cloud. They're like your personal computers, but they exist on the internet instead of being physically in front of you. You can create, customize, and use these computers for all different reasons, from running applications to hosting websites.

Pssst... EC2 = **Elastic Compute Cloud**.

Here's what the three words mean:

- 1 Elastic** = flexible. This service can easily adapt and change in size and power to fit your needs.
- 2 Compute** = computing power. EC2 provides virtual computers that can do various tasks, just like your personal computer.
- 3 Cloud** = available over the internet.

- Switch your **Region** to the one closest to you!



In your EC2 console, choose **Launch instance**.

💡 What are EC2 instances?

If EC2 is the service that provides virtual computers or servers, each instance is one of those computers or servers that gets produced.

Just like you can choose a computer with more memory or a faster processor when you buy a laptop, with EC2 instances, you can pick a virtual computer that fits what you need for your projects. You can customize your EC2 instance's CPU, memory, storage, networking capacity and more!

💡 What's the difference between a virtual computer and a server?

A virtual computer is basically the same as a normal computer... only, without the screen, keyboard, and other things that make it super easy to use. Without these physical elements, it becomes virtual! When a computer is virtual, you can access it from your own physical computer - basically giving your normal computer super powers.

Turns out, a server is also a computer. But a server is specifically designed to be used by a whole group of other computers to help with things like more power or storage. Servers are great for things like hosting websites, storing files, or running games that multiple people play together, whereas virtual computers are designed to be an extension of a personal computer.

- Let's set up your EC2 instance!
- In **Name**, enter the value.

network-prod-enter your name

- Choose **Add additional tags**, which is right next to your **Name** field.
- Choose **Add new tag**.
- For the next tag, use this information:
 - Key: Env
 - Value: production

💡 **Why are we creating a new tag? What does this tag mean, how will it be useful later?**

Tags are like labels you can attach to AWS resources for organization.

In this case, we're creating a tag called "Env" with a value of "production" or "development" to label the instances used in production vs development environments.

This tagging helps us with identifying all resources with the same tag at once (they are useful filters when you're searching for something), cost allocation, and applying policies based on environment types. You'll see the last point about policies in action soon!

The screenshot shows the AWS Management Console interface for launching an EC2 instance. The 'Name and tags' section is active, displaying two tags: 'Name' with the value 'network-prod-sak-shi' and 'Env' with the value 'production'. The 'Summary' section on the right provides a overview of the configuration: 1 instance, Amazon Linux 2023 kernel-6.1 A... AMI, t3.micro instance type, and a new security group. The 'Launch instance' button is prominently displayed in orange at the bottom right of the summary section.

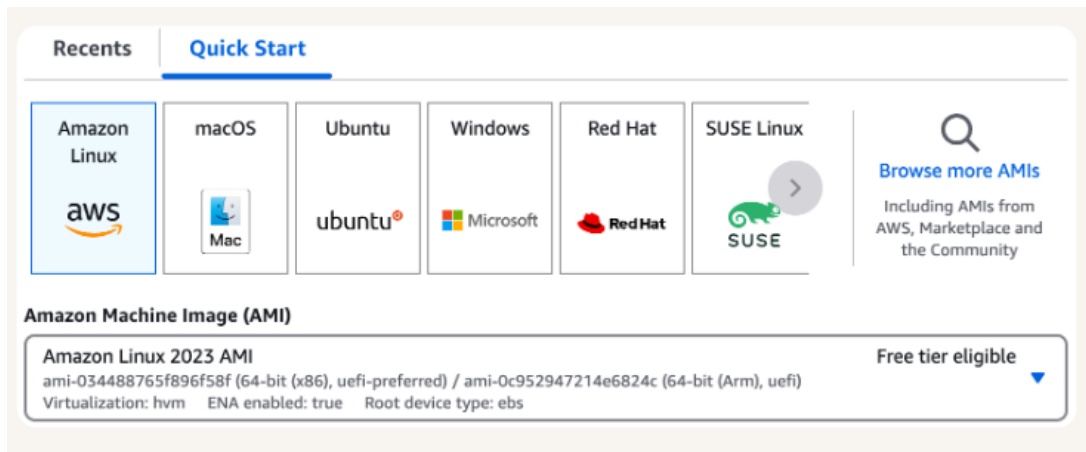
Head on down to see your EC2 settings and make sure the **Amazon Machine Image (AMI)** is using a **Free tier eligible** option.

💡 What is AMI? What is Free tier eligible?

When you buy a new computer off the shelf, most computers already have some software and the operating system (e.g. MacOS, Windows) already configured and set up for you!

AMI stands for Amazon Machine Image, and it's very similar to those pre-built computers. An AMI is a template or blueprint used to create EC2 instances and contains the operating system along with the applications needed to launch the instance.

Free tier eligible AMIs are those that qualify for the AWS Free Tier, so you won't get charged for using it.



For the instance type, also make sure you're using a **Free tier eligible** option!

💡 What is instance type?

If AMIs give you pre-built software and operating systems, instance types cover the **'hardware'** components.

CPU power, memory size, storage space and more!

So, while the AMI decides what operating system your server runs, the instance type determines how fast and powerful it performs.

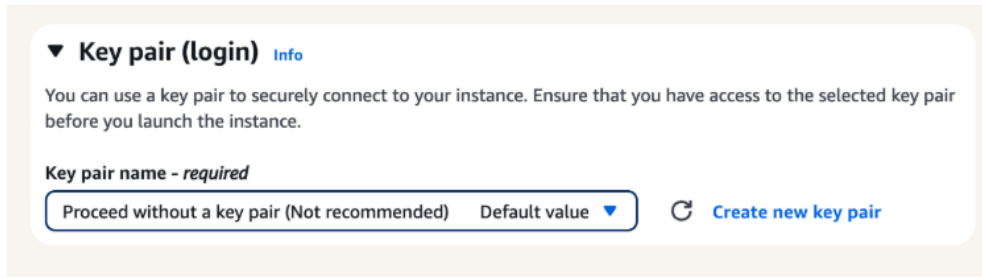
For **Key pair (login)**, select **Proceed without a key pair**.

💡 What is a key pair? Why does it say (Not recommended) next to proceeding without one?

A key pair is primarily used for accessing your EC2 instance securely **without going through the AWS Management Console**. Instead of the Management Console, you're using SSH (Secure Shell) Access with your key pairs - this is out of scope for this project, but you'll learn more about SSH and key pairs in a networking or compute-themed project!

Proceeding without a key pair means you won't have SSH (Secure Shell) access to your instance, which is generally not recommended because it limits your ability to troubleshoot or manage your EC2

instance through a secure way outside of the Console. It's always safer and more flexible to have a key pair set up, so you would create a key pair for bigger projects that you work on over a longer period of time.



▼ **Key pair (login)** [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Proceed without a key pair (Not recommended) Default value ▼ [Create new key pair](#)

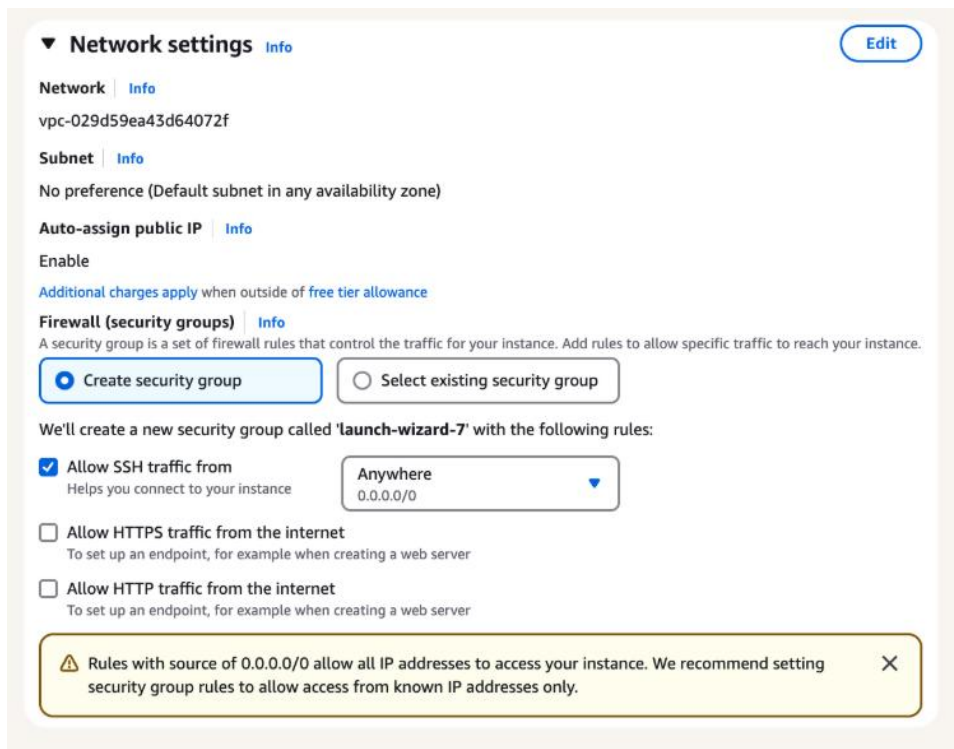
You're ready! Click **Launch instance**.

💡 Hold up... what are the settings we've skipped just now?

We skipped configuring network and storage settings for simplicity in this project. These settings are crucial for fine-tuning your EC2 instances' performance, security, and connectivity, but for this project, we'll focus on the basic steps of launching instances with minimal configuration.

Network settings define how your instances interact with the internet and other AWS resources, determining factors like IP addresses and network routing.

Storage settings involve choosing the type and size of storage volumes (like hard drives) that your EC2 instance will use to store data.



▼ **Network settings** [Info](#) [Edit](#)

Network [Info](#)
vpc-029d59ea43d64072f

Subnet [Info](#)
No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)
Enable
[Additional charges apply when outside of free tier allowance](#)

Firewall (security groups) [Info](#)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group ☐ Select existing security group

We'll create a new security group called 'launch-wizard-7' with the following rules:

☒ Allow SSH traffic from
Helps you connect to your instance
Anywhere
0.0.0.0/0

☐ Allow HTTPS traffic from the internet
To set up an endpoint, for example when creating a web server

☐ Allow HTTP traffic from the internet
To set up an endpoint, for example when creating a web server

⚠️ Rules with source of 0.0.0.0/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only. ✕

Looks like a success!

Now let's create one more EC2 instance for the **development environment**.

💡 What do the development vs production environments mean?

Development and production environments refer to different stages in the software development lifecycle.

The **development** environment is where developers write, test, and debug code before it's deployed to **production**, which is the live environment that your end users can use!

Repeat the same flow, but this time using these tags:

- Name: nextwork-dev-enter your name
- Env: development

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Key	Value	Resource types
Q Name	Q nextwork-prod-sak-sh	Select resource types
Instances		
Remove		

Add new tag

You can add up to 48 more tags.

Application and OS Images (Amazon Machine Image) [Info](#)

An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search

Summary

Number of instances

1

Software Image (AMI)

Amazon Linux 2023 kernel-6.1 A...[read more](#)

ami-01b6d88af12965bb6

Virtual server type (instance type)

t3.micro

Firewall (security group)

New security group

Cancel

Launch instance

Preview code

- Launch your second instance.
- Select **Instances** from your left-hand navigation panel.
- If you only see one instance on your page, make sure to use that refresh button!
- Let's have a look at your wonderful work.
- Select the checkbox next to one of your instances, and a popup window of information pops up!
- Select the **Tags** tab.

Instances (2) [Info](#)

Last updated 1 minute ago

Find Instance by attribute or tag (case-sensitive)

All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☐	nextwork-pro...	i-0d8e15d7c8a7ccf51	Running	t3.micro	3/3 checks passed	View alarms	ap-south-1a	ec2-65-0-94
☐	nextwork-dev-...	i-084a19f56d2461f37	Running	t3.micro	Initializing	View alarms	ap-south-1a	ec2-43-205

Voila - you'll see the tags you've defined right here.

Create an IAM Policy

🎉 WOOOOOOO! You've deployed two EC2 instances, one for your production environment and one for your development environment.

Now let's move into our second task as NextWork's engineer - it's time to onboard the team's new intern and **set up permission policies**.

Our intern should have permission to the development EC2 instance but **not** the production instance. We don't want them to accidentally shut down the platform or push their changes to the production environment while they're just testing things!

To start this task, we'll use AWS IAM to **give** our intern access to the development instance first.

In this step, get ready to: Create an IAM policy that gives access to the development instance.

Head to your IAM console.

WHAT IS IAM?

In Amazon Web Services (AWS), IAM is a service that helps you securely control access to your AWS resources. You use it to manage who can sign in and what permissions they have.

Now on the left-hand navigation panel of your IAM console, choose **Policies**.



What is a Policy?

An IAM policy is a rule for who can do what with your AWS resources. It's all about giving permissions to IAM users, groups, or roles, saying what they can or can't do on certain resources, and when those rules kick in.

- Choose **Create policy**.
- Switch your Policy editor tab to JSON.



What are we doing right now?

You can create and edit AWS policies in the visual editor or JSON. In this project, we will use the JSON method.

The screenshot shows the AWS IAM console interface during the 'Review and create' step of policy creation. The breadcrumb navigation at the top reads 'IAM > Policies > Create policy'. On the left, a sidebar shows two steps: 'Step 1: Specify permissions' and 'Step 2: Review and create', with the second step being the active one. The main content area is titled 'Review and create' with a sub-header 'Review the permissions, specify details, and tags.' Below this, the 'Policy details' section contains a 'Policy name' field with the value 'NextWorkDevEnvironmentPolicy' and a 'Description - optional' field with the value 'IAM Policy for NextWorks development environment'. At the bottom, the 'Permissions defined in this policy' section shows 'Explicit deny (1 of 450 services)'.

IAM > Policies > Create policy

Step 1
Specify permissions
Step 2
Review and create

Specify permissions [Info](#)

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

Visual | **JSON** | Actions ▾

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      },
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "*"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.
[+ Add new statement](#)

Policy editor

Visual | **JSON** | Actions ▾ 

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Action": "ec2:*",
7       "Resource": "*",
8       "Condition": {
9         "StringEquals": {
10          "ec2:ResourceTag/Env": "development"
11        }
12      },
13    },
14    {
15      "Effect": "Allow",
16      "Action": "ec2:Describe*",
17      "Resource": "*"
18    },
19    {
20      "Effect": "Deny",
21      "Action": [
22        "ec2:DeleteTags",
23        "ec2:CreateTags"
24      ],
25      "Resource": "*"
26    }
27  ]
28 }
```

Edit statement

Select a statement

Select an existing statement in the policy or add a new statement.
[+ Add new statement](#)

[+ Add new statement](#)

JSON Ln 29, Col 0 5851 of 6144 characters remaining

💡 Let's unpack this spicy policy

This policy allows some actions (like starting, stopping, and describing EC2 instances) for instances tagged with "Env = development" while denying the ability to create or delete tags for all instances.

💡 Extra for Experts: how are JSON policies structured?

This means 2012-10-17 is the date of the latest policy version. This tells you whether the policy is up to date with the latest standards and practices.

Statement

The main part of the policy structure and defines a list of permissions.

Effect

This can have two values - either **Allow** or **Deny** - to indicate whether the policy allows or denies a certain action. **Deny** has priority. Looking at the first statement, "Effect": "Allow" means this statement is trying to allow for an action.

Action

A list of the actions that the policy allows or denies. In this case, "Action": "ec2:*" means all actions that you could possibly take on EC2 instances are allowed. Woohoo!

Resource

Which resources does this policy apply to? Specifying "*" means all resources within the defined scope (see the next point).

Condition Block (optional)

The circumstances under which the policy is in action. In this case, the condition is that the resource is tagged **Env - development**. This means specifying "Resource": "*" in the line above means all resources with the **Env - development** tag are impacted by your statement.

- Select **Next** when you're ready.
- Fill in your policy's details:
 - Name: NextWorkDevEnvironmentPolicy
 - Description: IAM Policy for NextWork's development environment
- Oh no! Turns out there's a rule for the characters allowed in your Policy description. Edit this description to get rid of that error (can you tell which character is not valid? There's a hint given to you right underneath the **Description's** text box).
- Choose **Create policy** when you're done.

Create an AWS Account Alias

Alrighty, that was the permission policy all set up 

Now that we can give our intern access to the development instance, the intern can't wait to start. They'd love to jump into the team's AWS account right away!

Sounds great... where should they go to log in?

In this step, get ready to:

Simplify user login to your AWS account using an Account Alias.

- Still in your IAM console, select Dashboard from the left-hand navigation panel.
- In the right-hand side of the dashboard, choose **Create** under **Account Alias**.



What is an Account Alias? Why are we creating one?

Once you onboard new users into your AWS account (which we'll do for our new NextWork intern), these new users get access through a unique log-in URL for your account.

An Account Alias is a friendly name for your AWS account that you can use instead of your account ID (which is usually a bunch of digits) to sign in to the AWS Management Console.

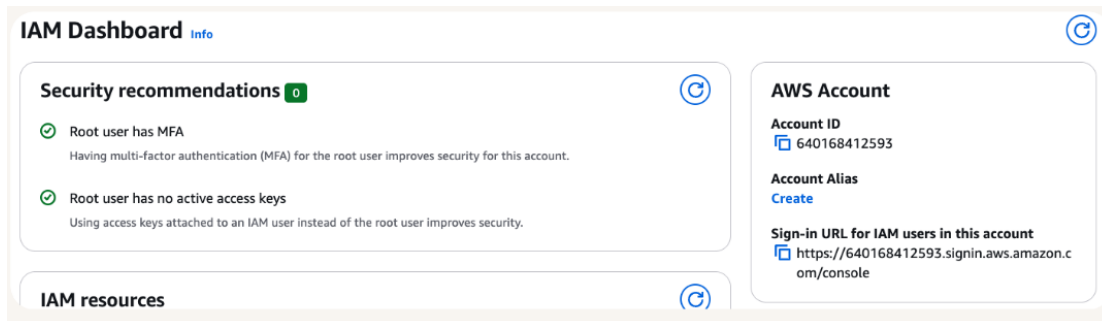
Your AWS account's sign-in page has this URL by default:

`https://Your_Account_ID.signin.aws.amazon.com/console/`

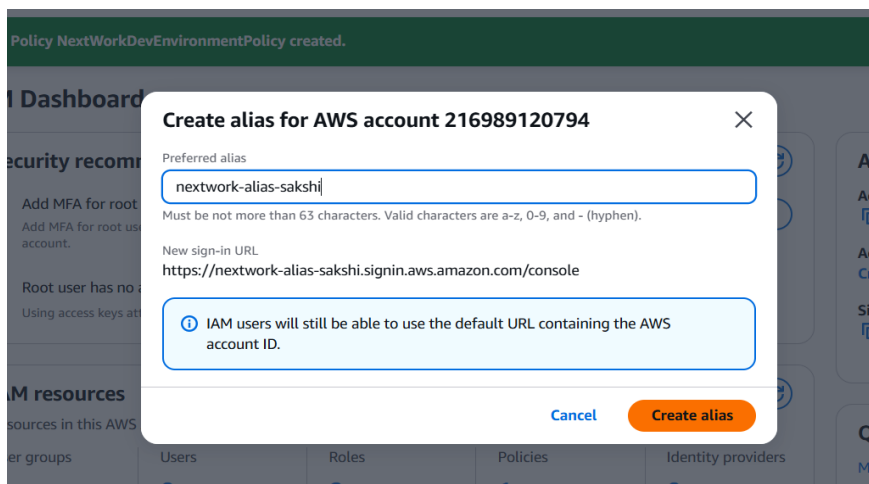
If you create an AWS account alias for your AWS account ID, your sign-in page URL looks more like:

`https://Your_Account_Alias.signin.aws.amazon.com/console/`

You would create an alias to make it easier to remember and share your AWS console's login URL with others e.g. NextWork's new intern. Companies often use this so that their AWS account sign-in page is more user-friendly for their users!



In the **Preferred alias** field, enter: nextwork-alias-enter your name



Choose **Create alias**.

Create IAM Users and User Groups

Your account alias is looking slick! It should be a lot faster for users to find your account and log in now.

Our new intern doesn't actually have a way to log in to the team's AWS account yet... what should their username and password be?

You wouldn't want to just share your account details with them. After all, you have access to the production instance, which they shouldn't get access to.

Let's solve this problem using IAM again - this time we're using two other tools called groups and users.

In this step, get ready to:

Set up a dedicated IAM **group** for all NextWork interns, so you can manage all interns' permissions from one place.

Set up a dedicated IAM **user** for your new intern, so they have a way to log in.

- Choose **User groups** in your left-hand navigation panel.
- Choose **Create group**.
- Let's create your first user group!

💡 What is an IAM user group?

An IAM user group is a collection/folder of IAM users. It allows you to manage permissions for all the users in your group at the same time by attaching policies to the group rather than individual users.

🔗 To set up your user group:

- Name: nextwork-dev-group
- Attach permission policies: NextWorkDevEnvironmentPolicy
- Select **Create user group**. Success!

Now let's add Users to your user group.

💡 Why do we need users in our user group?

IAM users are the people that will get access to your resources/AWS account, whereas user groups are the collections/folders of users for easier user management.

We're adding users to **nextwork-dev-group** to grant them the permissions associated with that group.

This simplifies managing permissions and ensures consistency across users who have similar access to AWS resources. Imagine if you have a whole team of 5 interns (users) that need the same permission settings next Summer!

- Choose **Users** from the left-hand navigation panel.
- Choose **Create user**.
- Let's set up this user! Under **User name**, enter: nextwork-dev-enter your name
- Tick the checkbox for **Provide user access to the AWS Management Console**.

💡 Why are we ticking this box?

If you don't tick this box, your new user won't get to sign in and access AWS services through the Console. They'll have to access AWS services through other, more advanced methods - we'll cover those advanced methods (e.g. AWS CLI, SDKs, APIs) in a future project!

💡 Ahem... in the real world, you should absolutely leave this box checked! We are leaving it unchecked because you'll have to create a new password for this user, which is irrelevant to our learning objectives for today.

💡 **Note:** This does not show up for every AWS Account, but if you see a highlighted pop-up that asks "**Are you providing console access to a person?**" - select **I want to create an IAM user**.

Shoutout to Andrew Lin in the community for mentioning this and sharing a screenshot! 🙌

User details

User name

nextwork-dev-maya

The user name can have up to 64 characters. Valid characters: A-Z, a-z, 0-9, and + = , . @ _ - (hyphen)

☒ **Provide user access to the AWS Management Console - optional**
If you're providing console access to a person, it's a [best practice](#) to manage their access in IAM Identity Center.

Are you providing console access to a person?

User type

☐ **Specify a user in Identity Center - Recommended**
We recommend that you use Identity Center to provide console access to a person. With Identity Center, you can centrally manage user access to their AWS accounts and cloud applications.

☒ **I want to create an IAM user**
We recommend that you create IAM users only if you need to enable programmatic access through access keys, service-specific credentials for AWS CodeCommit or Amazon Keyspaces, or a backup credential for emergency account access.

Console password

☒ **Autogenerated password**
You can view the password after you create the user.

☐ **Custom password**
Enter a custom password for the user.

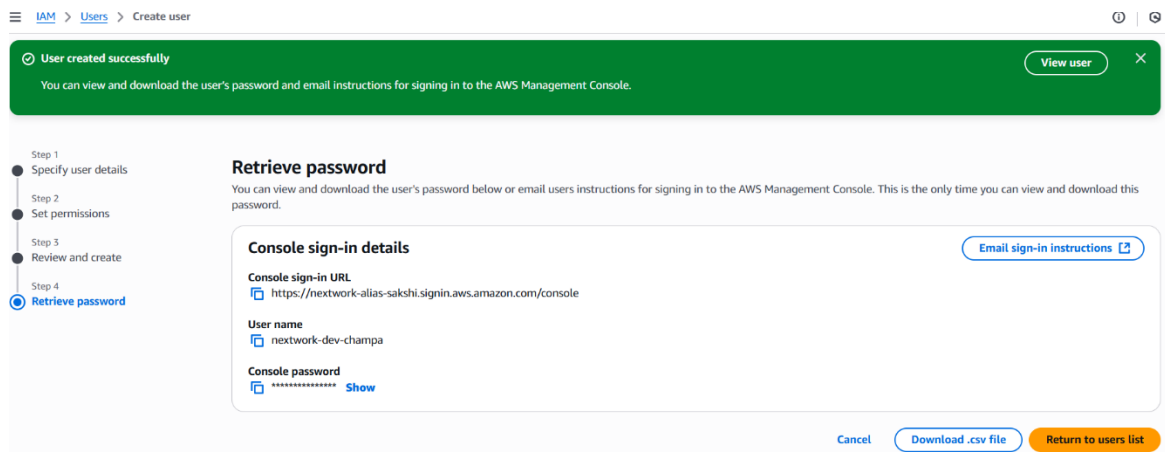
☐ Show password

☐ Users must create a new password at next sign-in - Recommended

- Select **Next** when you're ready!

- To set permissions for your user, we'll simply add it to the user group you've created. Select the checkbox next to **nextwork-dev-group**.
- Select **Next**.
- Select **Create user!**

WOOO it's a success - now you're seeing some specific sign-in details for your new user. Stay on this page.



Test your intern's access

The new intern is going to be stoked to receive their keys to the NextWork AWS account - well done 🥳

Before we pass them their login details, let's test the interns' IAM User's access first. That way we can make sure that they have the right access to our development instance (and not the production instance).

In this step, get ready to:

Log into AWS using the intern's IAM user.

Test the intern's access to your production and development instance.

- Copy the **Console sign-in URL**. Do not close this tab!
- Open a new **incognito window** on your browser.
- Open the new console sign-in URL in your incognito window.
- Using the **User name** and **Console password** given in your IAM tab, let's log in!

As a new user, you'll notice that some of your dashboard panels are showing **Access denied** already.

- Head to your **EC2** console, and make sure you're in the same **Region** as the one where you deployed your two production and development instances.
- Head to **Instances**.
- Select your **production** instance, and in the **Instance state** dropdown, select **Stop instance**.

Instances (1/2) Info

Last updated about 4 hours ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

nextwork-dev...

i-07961fb09f5fa1a8c

Running

NextWorkCod...

i-075da95b5b2ff4e05

Running

Status check

Alarm status

Stop instance

Start instance

Reboot instance

Hibernate instance

Terminate (delete) instance

2/2 checks passed

View alarms

Let's try to stop this instance. Select **Stop**.

💡 Yoikes! At the top of your page, an angry-looking banner tells us we've failed to stop this instance. The banner tells us it's because we're not authorized! We don't have permission to stop any instance with the production tag.

Failed to stop the instance i-0c9320aa130ea0f43

Diagnose with Amazon Q

You are not authorized to perform this operation. User: arn:aws:iam::640168412593:user/nextwork-dev-maya is not authorized to perform: ec2:StopInstances on resource: arn:aws:ec2:ap-southeast-2:640168412593:instance/i-0c9320aa130ea0f43 because no identity-based policy allows the ec2:StopInstances action. Encoded authorization failure message:

ilfdLoeBzLf0NSprCXyCJE1I123QJe3_4HZSdMGO9hZWR-
fEa10hjK6sAcnDmRribuePzHQpYQBKhKDJIvdovF8Z13e4mgFFDLJu45wFOOrMnBLrCtyMTfbFWtcxMUJSHGrDFGeNWqMzf_UbB6
YQmjMZICjMKBBXolw9TLBr68KckpHFqBgGbxPh7qtRi0TcSi0RHD32aZFqY6XMDc8HJFyFCWp1XCd4wTYm9p0jk5qxW4YbaJ2z9Jt-
IMQP41rg54661agBhngxRbfNarUzgBU6F4GA7k6vel5ln3OKLFD30sPLIDoKCNKIQbFxxIq9aczLz8ZIU-
UyFk1QFMNBwHv7XMcZXgz2ebjXmTZ5menftxUCpf_MIPBbSwmMTmPjZzPQxDtVA_I8DX57f2VUZ0oDSRNNzQqcmkj015mQHb
NYg1fPsJNRZtd2m39GmqZJylc-xsX-
satVk688fQvUanEHpgrabVMnmCDssZe4VoLfxhqlforBGgKWQuV_o6NG1d7b9fyb34iv99Cg9Hhpb8UJgxkyP8SRtOwxtla1QfSo
DE4PGxAy2xdVvYxDAQNKvt8AP1QVFUJndstNOTI2Xcmz8dSDkD9CSKV25OHVZPwn4kFD5lkWRO_cpVrqelKvFxcCZS0drbAPvrH8Uz
WO5EIHfAbjuYUigxj3f55XikBolxmQPntSsBUCIH-I4JgKtGC3fHT9u8Vt0ghB3R81S4pAzt7HISPK2QoEukw-
4HoRpWdgp0Rk_LtkACs6uioTkB1HpmiTFZMrI7aNw-Tw-
jbcUx9E0YVQdyILAwGAqipvZCnqEzbiNFLwe9bbLBqljTJ9sPcuSVDpXmrXDdu1It-WV-
yJ8JWvgxHwaE7AFXg4JXhQH9RWnWND4zud762L3-Hq